

1. What is HBase?

HBase is an **open-source, distributed, column-oriented NoSQL database** that runs on top of **Hadoop HDFS**.

It is designed to handle **large volumes of sparse data** — meaning datasets with many columns where most cells are empty (like time-series data, logs, IoT data, clickstreams, etc.).

HBase is based on **Google Bigtable**, a paper published by Google in 2006, and was developed as part of the **Apache Hadoop ecosystem**.

Why Do We Need HBase?

♦ Problem with HDFS (alone)

HDFS is great for **storing large files** and **batch processing**, but:

- It **does not allow random reads/writes** — you can't easily update a single record.
- It is optimized for **sequential access**, not for quick lookups.
- You must process the entire file, even if you need one row.
- No **real-time access** — it's more for analytics and MapReduce jobs.

So, if you want **real-time read/write access** to data stored in Hadoop — you need **HBase**.

3. What is HBase Used For?

HBase is used when you want:

- **Real-time random read/write access** to very large datasets.
- To **store unstructured or semi-structured data** (no fixed schema like RDBMS).

- To **scale horizontally** (add more servers to handle more data).
- To manage **time-series or sparse data** efficiently.

Typical Use Cases:

Use Case	Description
Facebook Messaging / WhatsApp	Storing billions of messages per user
Time-Series Data (IoT, Logs)	Billions of sensor readings
Recommendation Systems	Store and fetch user activity logs
Search Indexing / Web Crawling	Huge dataset, random access
E-commerce Product Catalogs	Millions of products, dynamic attributes

4. How HBase Works (Conceptually)

HBase provides **table-like storage**, but internally it's very different from an RDBMS.

Key Concepts:

Concept	Description
Table	Logical container for data (like in RDBMS)
Row Key	Unique identifier for each record (sorted lexicographically)
Column Family	Logical grouping of columns (defined at table creation)
Column Qualifier	Actual column name inside family
Cell	Intersection of row + column + timestamp
Timestamp	Each cell stores multiple versions (based on timestamp)

Example:

Row Key	personal:name	personal:email	marks:math	marks:english
roll_101	Rahul	rahul@gmail.com	95	87

Explanation:

- **Row Key:** roll_101
 - **Column Families:** personal, marks
 - **Column Qualifiers:** name, email, math, english
 - **Each cell** can have multiple **versions (timestamps)**.
-

5. HBase Architecture (Deep Dive)

Let's break it down component by component.

1. Client

- Connects to HBase to read/write data.
- Communicates through the HBase API or Shell.

2. HMaster

- Controls overall cluster.
- Manages:
 - Region assignment to RegionServers.
 - Schema changes (create/delete tables).
 - Load balancing (splitting and moving regions).

- But note: **HMaster is not in the critical data path** — even if it fails, RegionServers continue working.

HBase has **two main types of nodes**:

1. **HMaster**

- Coordinates RegionServers (assigns regions, monitors health).
- Manages schema changes (create, delete tables).
- Handles load balancing and failover.
- But **does NOT handle client reads/writes** directly.

2. **RegionServer**

- Actually stores data (in regions, HFiles, MemStore, etc.).
- Handles all client read/write requests.

♦ **What "Not in the critical data path" Means**

It means that **client read and write requests do not go through the HMaster**.
The HMaster only manages **metadata and coordination**, not actual data I/O.

♦ **Example: Client Read/Write Workflow**

Scenario:

Let's say you have an HBase table `Student` with 10 million records.
The table is split into **regions**, each hosted by different RegionServers.

Region	Key Range	RegionServer
--------	-----------	--------------

Region-1	A–F	RS1
----------	-----	-----

Region-2	G–L	RS2
----------	-----	-----

Region-3	M–R	RS3
----------	-----	-----

Region-4	S–Z	RS4
----------	-----	-----

Step 1: Normal Operation

Client writes data:

```
Put("Rahul", {name: "Rahul", marks: 90})
```

1. The **client asks ZooKeeper** where the META table is.
2. It contacts the **META table** to find out which RegionServer holds “Rahul”.
3. META says → “Rahul” (R) belongs to **Region-3** on **RS3**.
4. Client directly connects to **RS3** and sends the write request.
5. **HMaster is not involved at all** in this data flow.

Same for reads:

```
Get("Rahul")
```

Client directly queries RS3 again.

Step 2: HMaster Fails

Now suppose suddenly the **HMaster crashes** due to a hardware fault.

What still works:

- Clients still know which RegionServer has their region.
- They continue to **read/write directly** to RegionServers.
- RegionServers still flush MemStores, write to HFiles, update WALs, etc.
- No data loss or downtime occurs for ongoing reads/writes.

What doesn't work temporarily:

- Creating or deleting tables.
- Assigning new regions (if a RegionServer dies).
- Balancing load between RegionServers.

So, **existing RegionServers keep serving data**, but new administrative operations wait until the HMaster recovers.

Step 3: HMaster Recovery

- A **backup HMaster** (standby) automatically takes over using ZooKeeper.
- It restores metadata, resumes monitoring, and continues normal cluster management.
- The clients never notice any interruption in their data access.

3. RegionServer

- Handles read/write requests from clients.
- Each RegionServer manages multiple **Regions**.
- Stores data in **MemStore** (RAM) and **HFiles** (on HDFS).
- Responsible for **flushes, compactions, and region splitting**.

4. Region

- A subset of a table (continuous range of row keys).
- When a Region grows too big, it splits into two.
- Helps in parallelism and scalability.

5. ZooKeeper

- Keeps track of RegionServers.
- Helps with leader election and failover.
- Clients use ZooKeeper to find which RegionServer holds which Region.

6. HDFS

- The underlying file system for permanent storage.
 - Stores the actual **HFiles** and **Write-Ahead Logs (WALs)**.
-

6. Data Flow Inside HBase

Write Path (Put Operation):

1. Client sends data to RegionServer.
2. RegionServer writes to **WAL (Write Ahead Log)** → ensures durability.

3. Data written into **MemStore** (in-memory cache).
4. When MemStore full → flushed to **HFile** on HDFS.
5. Multiple HFiles → periodically merged through **compaction**.

Flow:

Client → RegionServer → WAL → MemStore → HFile (HDFS)

Read Path (Get/Scan Operation):

1. Client sends read request to RegionServer.
2. RegionServer checks:
 - **MemStore** (latest data)
 - **BlockCache** (frequently read data)
 - **HFiles** (older data on HDFS)
3. Data is combined and returned to the client.

Flow:

Client → RegionServer → (MemStore + BlockCache + HFiles) → Client

7. Key Features of HBase

Feature	Description
Column-Oriented Storage	Data stored by columns, not rows — efficient for sparse data
Scalability	Add more RegionServers to scale horizontally
Versioning	Multiple versions of a cell (by timestamp)
Real-Time Access	Fast read/write on massive datasets

Automatic Sharding	Table split into regions automatically
Integration	Works with Hive, Pig, Spark, MapReduce

8. How HBase Differs from RDBMS

Feature	RDBMS	HBase
Data Model	Relational	Column-Family based
Schema	Fixed	Flexible
Transactions	ACID	Eventually consistent (per row strong consistency)
Query Language	SQL	No SQL (API-based)
Storage	Row-oriented	Column-oriented
Scalability	Vertical	Horizontal
Access	Structured, limited	Random read/write, real-time
Ideal Use	Structured data	Big, sparse, semi-structured data

9. Integration in Big Data Ecosystem

Tool	Integration
Hive	SQL-like queries using HBase as backend
MapReduce	Process HBase data in distributed manner
Spark	Real-time analytics with in-memory processing
Pig	Data transformation using HBaseStorage handler
